MeDUSA SOP

Shakira de Geus, Mai Nguyen, Anouschka van der Zeeuw, Grace Huang, Ahmed Ali, Dirk Wevers

March 5, 2026

General

1 Getting started

1.1 On a Windows PC

All information regarding MeDUSA can be found via this link. When using MeDUSA for the first time on a Windows PC, the easiest way to do this requires the download of Docker and registration for this platform. As soon as this has been completed, one needs to download Windows Subsystem for Linux (WSL); this can be done through Docker via a screen that automatically pops up. Subsequently, in Docker, one can look up "thefollyllama/medusa": first, one needs to "pull" MeDUSA, and then it can be "run".

Once MeDUSA has been pulled and run, one can start it from the terminal: go to the folder on your PC that you want to process using MeDUSA, right-click and select the option "Open in Terminal". When this is done, Windows Powershell will open. Subsequently, in Windows Powershell, the following two lines of code should be run consecutively.[A] In the first line, one can specify a version number by writing ":v2", to run the second version, or ":latest" to run the latest version. By default, the latest version is run. Alternatively, the aforementioned lines can be run in the terminal in Docker, this has the same effect.

```
docker pull thefollyllama/medusa
docker run -e PASSWORD=medusa -p 8787:8787 -v .:/home/rstudio/local thefollyllama/medusa
```

Subsequently, one can navigate to one's web browser, and insert the following URL: "http://localhost:8787/". If all steps have been executed correctly, RStudio Server is opened, and you are asked for a username and a password; these are "rstudio" and "medusa", respectively. RStudio is opened; generate a new .R file for your analysis, and you're ready!

1.2 On a MAC

First, download "Docker Desktop" software from docker.com, which is required for pulling the Dockerfile and eventually accessing MeDUSA. Once the download is complete, open and run the Docker Desktop software on your PC. Create a folder on your PC with all the desired files to run on MeDUSA. Once the folder is created, right-click on the folder and choose "New Terminal at Folder". Once the terminal is open, copy and paste the first code into the terminal and then press Enter on your keyboard to run the first code, which by default uses the latest version.[A]

```
docker pull thefollyllama/medusa
```

Alternatively, to run a specific version, the first code can be modified to that specific version. For example, to run version 3, ":v3" should be written at the end of the code. See the following example:

```
docker pull thefollyllama/medusa:v3
```

After the first code has been run, copy and paste the second code into the terminal and then press Enter on your keyboard.[A]

```
docker run -e PASSWORD=medusa -p 8787:8787 -v .:/home/rstudio/local thefollyllama/medusa
```

[A] When copying this code, make sure not to include any additional spaces.

After running the two codes successfully, open and type in your web browser the following URL: **http://localhost:8787/**. This link will take you to the login page of RStudio Server, where you can log in with the username: **rstudio** and the password: **medusa**. The RStudio environment will open after logging in, where you can work with your data.

1.3 Using Ubuntu

MeDUSA can be run with Ubuntu version 24.04.1 with docker installed. Information on how to install docker can be found here. Next, the MeDUSA image needs to be downloaded from dockerhub. This can be done with the following code.

```
docker pull thefollyllama/medusa
```

By default, the latest version of MeDUSA is installed. Alternatively, to run a specific version, the first code can be modified to that specific version. For example, to run version 2, ":v2" should be written at the end of the code. See the following example:

```
docker pull thefollyllama/medusa:v2
```

Once MeDUSA has been pulled and run, it can be started from the terminal. The docker environment will run from the current directory in the terminal. Make sure that the current directory contains the data to be used in MeDUSA as the docker environment cannot access files outside the directory it is run from. MeDUSA can be run using the following line.

```
docker run -e PASSWORD=medusa -p 8787:8787 -v .:/home/rstudio/local thefollyllama/medusa
```

The name of the container is version-specific. If no version specification is given when pulling the image, no version needs to be added in the run command. (An example is given above). If a specific version is pulled, for example, thefollyllama/medusa:v2, the name used in the run command must contain:v2. See the code below on how to specify the version.

```
docker run -e PASSWORD=medusa -p 8787:8787 -v .:/home/rstudio/local thefollyllama/medusa:
    v2
```

Subsequently, one can navigate to one's web browser, and insert the following URL: "http://localhost:8787/". If all steps have been executed correctly, RStudio Server is opened, and you are asked for a username and a password; these are "rstudio" and "medusa", respectively. RStudio is opened; generate a new .R file for your analysis, and you're ready!

2 Working with MeDUSA

2.1 Getting started with MeDUSA

It is practical to set the working directory in RStudio to the directory that has all the required files. This can be done in the bottom-left panel: click on "More", and then "Set As Working Directory". Subsequently, to make use of MeDUSA, the MeDUSA library needs to be loaded. Furthermore, a library to read .xlsx files is required to read the metadata file. Assuming the metadata file is located in the working directory, this can be done by running the following code.

```
library(MeDUSA)
library(readxl)

metadata <- read_xlsx('metadata.xlsx')
```

2.2 Extraction

The first part of MeDUSA is the extraction of mzml files. Two alignments are necessary for comparison of data. The first alignment compares data in one file, across multiple scans. This alignment removes the time dimension and creates one intensity per m/z for each file. The default is to generate a mean of the intensity per m/z. A missingness threshold is set so that an m/z must occur in a set percentage of scans in order to not be filtered out as an artifact. The tolerance for an m/z to be considered the same feature across

multiple scans is referred to as the mzBinTolerance. If m/z values within the bin differ, the mzBinningMath will determine which method is used to determine the assigned m/z value. A second alignment compares the data in multiple files. This step aligns m/z values in multiple files, the intensity does not change further.

To extract a single mzml file the function `mzml_extract_file` can be used. This function extracts a single mzml file in either positive or negative mode or all scans simultaneously. The function takes the path to the mzml file and the polarity to extract. The polarity can be set to 1 = positive, 0 = negative, or "" = everything. Here is an example usage of extraction in positive mode:

```
mzml_extract_file(file, polarity = 1)
```

By default this function performs binning followed by missingness and low-intensity filtering. These are controlled by the magic parameter. If magic is set to FALSE no filtering or binning will be performed and only the centroided m/z values for each scan will be returned. Below is an example use of the magic parameter.

```
mzml_extract_file(file, polarity = 1, magic = F)
```

To extract and align multiple mzml files the function `mzml_extract_magic` is used. This function extracts multiple mzml files in the both polarities and performs filtering and time squashing (removal of time dimension). The input mzml files will be aligned twice: first across scans and then across files. The mzml files can be given as a list or by setting the working directory to the location of the files. Below are two example usages.

```
# list of files
files <- list.files(path_to_files)
mzml_extract_magic(files)

# working directory
setwd(path_to_files)
mzml_extract_magic(getwd())
```

It is possible to control the missingness and intensity thresholds used in the filtering as well as the bin tolerance, the method used to calculate the binned m/z and the time squash method. These can be set when using either extraction function by using the params parameter. Below is an example usage.

```
mzml_etract_magic(files, params = list(prebin_method = mean, missigness_threshold = 2,
    intensity_threshold = 100, timeSquash_method = max)
```

See the MeDUSA documentation for more information on different options.

2.3 Quality check

This part outlines the steps to check the quality of the data. A quality check should be performed on the entire data set before removing any samples. If a measurement is confirmed to be unreliable after inspecting the raw files as well, it is recommended to exclude it from the data set.

The `mz_quality_metrics` function helps to identify outliers. It frames the median, minimum, and maximum m/z values, as well as the total number of peaks and the number of peaks exceeding thresholds of 1,000, 10,000, and 100,000. `mzmetrics_quality_plot_all` generates plots for all these statistics. In the console, a new directory will be created with a timestamp where the plots will be saved.

The function `mz_quality_meta_check` checks whether the data files align with the metadata file. All mzObj columns must be found in metadata`$sample_name$` and *vice versa*. The metadata allows for easy identification of the samples. This dataframe should at least include `sample_name`, type, phenotype and `filtered_out`.

Below is an example where the data doesn't match the metadata. After running the `mz_quality_meta_check`, the following error appears in the console; here 'out' is the datafile and 'metadata' is the metadata file.

```
Error in mz_quality_meta_check(out, metadata):
  ERROR: MeDUSA::mz_quality_meta_check: MetaSamples not in data: neg_sumr_hep_013
```

```
ERROR: MeDUSA::mz_quality_meta_check: DataSamples not in meta:
    neg_sumr_hep_012_20230114112938
```

The code to resolve this essentially adjusts the sample name:[B]

```
colnames(out)[colnames(out)== "neg_sumr_hep_012_20230114112938"] <- "neg_sumr_hep_013"
```

If it is concluded that a bad measurement occurred, and the measurement should be removed from the dataset, the following code can be used:

```
#In this example 3 measurements were deemed 'bad' and were removed.

df_neg <- mzL_n %>%
  select(-c("neg_sumr_media_100",
            "neg_sumr_hek_061",
            "neg_sumr_media_147")) %>%
  filter_at(vars(-matches("mz")), any_vars(. > 0))

df_pos <- mzL_p %>%
  select(-c("pos_sumr_media_100",
            "pos_sumr_hek_061",
            "pos_sumr_media_147")) %>%
  filter_at(vars(-matches("mz")), any_vars(. > 0))

metadata$filtered_out[grep("100|861|147", metadata$sample_name)] <- TRUE
```

The functions `mz_quality_metrics`, `mzmetrics_quality_plot_all` and `mz_quality_meta_check` mentioned above are captured within the `mz_quality_magic` function.

2.4 Filtering

This part outlines the steps for processing metabolomics data using a set of functions related to subtraction, mass defect analysis, isotope tagging, and m/z filtering. These steps are crucial for removing noise, contaminants, and unwanted signals from the data, ensuring that the remaining data accurately represents the metabolites of interest in the samples.

(1) The subtraction process aims to isolate the cellular signal from the signals originating from *e.g.* cell media and ionization solvent. By comparing different types of samples (*e.g.*, cell, and media) and removing data that does not meet a specified threshold, we can eliminate the media signal from the dataset and retain relevant m/z values for the cells. The `mz_subtraction` function is used for this purpose. Here is an example of usage:

```
# Filter the input MZ-Obj to get the relevant sample and subtract objects; F means no
    subtraction
sample_mz_obj  <- MeDUSA::mztools_filter(input_mzObj, metadata, "blanks", "type", F)
subtract_mz_obj <- MeDUSA::mztools_filter(input_mzObj, metadata, "blanks", "type", T)
# Perform the subtraction with a threshold of 5
mz_subtraction(sample_mz_obj, subtract_mz_obj, threshold = 5 )
```

The `mztools_filter` function takes an input mz object, metadata, filter value, filter name, and a boolean. The filter value is used to select the sample type which is filtered out and can be found in the column specified by the filter name in the metadata. By setting the boolean, the selected filter value can be set to include (TRUE) or exclude (FALSE) the signals from this sample type.

(2) The mass defect analysis is used to remove artifacts introduced by salt clusters in the sample, assuming electrospray ionization. This is done by applying the method described by McMillan *et al.* [1] to zero out intensities that are lower than the calculated mass defect values. The `mz_mass_defect` function is used for this analysis. Here is an example of usage:

[B] If the same file name is used twice when doing measurements on the Orbitrap, a timestamp is appended to the file name of the second measurement. In this case however, the sample name should have been adjusted.

```
# Run the mass defect analysis with default parameters and generate a plot
mz_mass_defect(input_mz_obj, plot = TRUE, magicNumber1 = 0.00112, magicNumber2 = 0.01953)
```

(3) Isotope tagging is used to identify m/z values that are likely isotopes rather than unique features in HRMS data. The `mz_tag_isotope_hunter` function analyzes all m/z features and returns a list of m/z values that are isotopes of each other, allowing the user to decide whether to remove these values if desired. The `input_mz` can be a full mz object or only the mz column and the default isotope target is ^{13}C. Here is an example of usage:

```
# Hunt for isotopes in the input MZ-Obj with default parameters
mz_tag_isotope_hunter(input_mz, iso_target = 1.0034, iso_iter = 5, tol_ppm = 5e-6, cores =
    4)
```

(4) m/z filtering functions. The blacklist filtering function is used to remove m/z values that have been previously identified as contaminants from the dataset. This helps clean the data and reduce noise before statistical analysis. The blacklisted mz values can be given as a list or vector, string of the csv file name, containing name with corresponding mass or set to default by providing an empty string. Here is an example of usage:

```
# Filter the input MZ-Obj to remove blacklisted m/z values
mz_filter_blacklist(input_mz_obj, blacklist = "", tolerance = 5e-6)
```

The missingness filtering function removes m/z values that have a zero value in a specified percentage of the samples. This helps eliminate features that are likely noise or artifacts. The percentage can be set with the parameter threshold as decimal (percentage) or integer (number of samples) Here is an example usage:

```
# Filter the input MZ-Obj to remove m/z values with a zero value in more than 10% of the
    samples
mz_filter_missingness(input_mz_obj, threshold = 0.1, msg = "")
```

The low-intensity filtering function filters out noise by eliminating low-intensity values. The user can set a threshold for the minimum intensity, and any intensity below this value will be set to zero. Here is an example of usage:

```
# Filter the input MZ-Obj to set intensities below the threshold to zero
mz_filter_low_intensity(input_mz_obj, threshold, msg = "")
```

The `mz_filter_magic` function applies all the above filtering steps (minimum intensity, missingness threshold, and blacklist) in one go. It helps streamline the data-cleaning process before statistical analysis. Here is an example of usage:

```
# Apply all filters with specified minimum intensity and missingness threshold
mz_filter_magic(input_mz_obj, min_intensity, missingness_threshold = 0.1, blacklist =
    FALSE)
```

(5) Please note that proper parameter selection is crucial for each function, and users should carefully consider the nature of their data and research questions when applying these filters. Additionally, it is recommended that the parameter values used for each analysis be documented for reproducibility purposes.

2.5 Post-processing and statistics

Many of the key terms in bulk metabolomics are also discussed in the work by Boccard *et al.* [2] After the filtering has been completed, MeDUSA offers several possibilities to find the key determinants in your data. Several possibilities are provided for this purpose. First, the data needs to be post-processed. First, using the function `mz_post_imputation`, values are imputed: zeroes are replaced by random values below the noise threshold, as not all data processing techniques can handle zeroes. Subsequently, the data matrix is normalized by applying the function `mz_post_normalization` which uses probabilistic quotient normalization (PQN). This function also takes the metadata file as an argument: one of the samples is taken as a reference. As some data analysis techniques require the data to be log-transformed, the function `mz_post_log` is provided. The function `mz_post_pivot_longer` does not do anything to the data, but rather rearranges it: the data frame is converted to a frame consisting of three columns, each row listing the sample name, m/z value and corresponding intensity. Lastly, the function `mz_post_magic` combines all aforementioned post-processing functions, including the log transformation; therefore, it also requires the

metadata file to be given, as part of the PQN.

After post-processing, the key-determining features can be identified in the data. A straightforward example is the `mzlog_analysis_welch` function, which applies Welch's t-test to the data. To this end, the data frame needs to be split into two separate data frames, each representing a different phenotype. Next to the two data frames, the function also takes an argument `adjust_method` that defines whether the p-values resulting from the t-test should be adjusted to control the false discovery rate (FDR). An example is given in the code below. If no p-value adjustment is required, `adjust_method` should be set to `FALSE`.

```
res <- mzlog_analysis_welch(df_A, df_B, adjust = 'fdr')
```

The function `mzlog_analysis_fold` returns the fold change per m/z value, given two different phenotypes. By default, the mean of one phenotype is divided by the mean for the other phenotype. Generating a principal component analysis (PCA) [3] plot for the data is a suitable way to see whether any samples are more similar to each other in the dataset, *e.g.* because they are part of the same subpopulation. This can be done using the function `mzlog_analysis_pca`, which only returns plots. This function also takes the metadata as an argument, so the data points in the PCA plot - each representing a sample - can be colored according to phenotype. Furthermore, a blacklist of m/z values that should not be considered when making the plots can be passed using the argument `sample_blacklist`.

The functions `mzlog_analysis_heatmap` and `mzlog_analysis_volcano_magic` can be used to generate a heatmap and a volcano plot for the data, respectively. The `heatmap` function, next to the data and metadata file, also takes the arguments `annotation`, `save_file` and `cluster`: these take a column name from the metadata to base the annotation on, a path to save the figures to, and a Boolean to determine whether the data should be clustered in the heatmap. The `volcano_magic` function integrates the t-test and the generation of a volcano plot and only takes two data frames, each representing one phenotype.

Lastly, a useful tool in the identification of features predictive for sample class is random forest (RF) modeling. RF models, however, work best when strongly correlated features are removed. To this end, the function `mzlog_rf_correlation` can be used; it only takes the post-processed data and a correlation threshold as input. After the removal of the most correlated features, the number of features used to build the RF model needs to be optimized. This can be done using the `mzlog_rf_select` function. This function takes the output of the `select` function, the metadata file, and a sequence of values (`feat_size_seq = `) from which the optimum number of variables to be included in the RF model is selected.[C] Lastly, using the function `mzlog_rf`, the optimized RF model is built. This function takes a variety of input values, including the post-processed data, the metadata file, and the output of the previous function. This is also elaborated upon here. All in all, the code looks like this.

```
res_decor <- mzlog_rf_correlation(data_post)
res_select <- mzlog_rf_select(res_decor, metadata, feat_size_seq = seq(50, 1000, by = 50))
rf_model <- mzlog_rf(data_post, metadata, res_select)
```

The `mzlog_rf` also takes seed values used for model construction; these can also be varied from analysis to analysis to structurally ensure randomness.

References

1. McMillan, A. *Utilizing Untargeted Metabolomics to Characterize Microbial Communities and Identify Biomarkers of an Unhealthy State* ISBN: 9798841714194. PhD thesis (The University of Western Ontario (Canada), Canada – Ontario, CA, 2016).
2. Boccard, J., Veuthey, J.-L. & Rudaz, S. Knowledge discovery in metabolomics: An overview of MS data handling. *J. Sep. Sci.* **33,** 290–304. ISSN: 1615-9314 (2010).
3. Bro, R. & Smilde, A. K. Principal component analysis. *Anal. Methods* **6.** Publisher: Royal Society of Chemistry, 2812–2831. `www.rsc.org/methods` (May 2014).

[C] Depending on the choice of the sequence of values, this optimization step can take rather long.